Side Activity 9: Aggregation, GROUP BY & HAVING

CSCI3170 Introduction to Database Systems

Learning Objectives

In this Side Activity, you will learn:

- How to use **aggregate functions** (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`) to summarize data.
- How to use `GROUP BY` to compute aggregations **per group** (e.g., per department).
- How to use `HAVING` to filter groups based on aggregate conditions.
- The critical difference between `WHERE` (filters rows) and `HAVING` (filters groups).
- How to combine `JOIN`, `WHERE`, `GROUP BY`, `HAVING`, and `ORDER BY` in a single query.

Part 0: Getting started (University schema dataset)

Please refer to the instructions in **Side Activity 3** to access your Oracle environment (SQL*Plus).

Step 0: Copy the Side Activity files to the server (same environment as Side Activity 3)

In Side Activity 3 (see "Part 1: Connecting to Oracle"), you SSH to `linux1.cse.cuhk.edu.hk` and then connect to SQL*Plus. If your `DDL.sql` and `smallRelationsInsertFile.sql` are on your **local computer**, you must copy them to the server first.

```
# Run on your LOCAL computer (macOS/Linux)
cd /path/to/CSCI3170_Tut
scp DDL.sql smallRelationsInsertFile.sql \
  {your_cse_id}@linux1.cse.cuhk.edu.hk:~/

# Run on the SERVER after SSH login (optional check)
ls -l DDL.sql smallRelationsInsertFile.sql
```

File location matters

In SQL*Plus, `@DDL.sql` runs a script from your **current working directory** on the server. If SQL*Plus says it cannot find the file, check where you copied it (`pwd`, `ls`) and run SQL*Plus from that folder.

Step 1: Create tables

Run the DDL script (tables: `student`, `instructor`, `course`, `section`, `takes`, `teaches`, etc.).

```
@DDL.sql
```

Step 2: Insert sample data

Load the small dataset.

```
@smallRelationsInsertFile.sql
```

Step 3: Quick sanity checks

```
SELECT COUNT(*) AS num_students   FROM student;
SELECT COUNT(*) AS num_courses    FROM course;
SELECT COUNT(*) AS num_sections   FROM section;
SELECT COUNT(*) AS num_takes_rows FROM takes;
```

Part 1: Aggregate Functions & GROUP BY

Key Concept: Aggregate Functions

An aggregate function takes a **set of rows** and returns a **single value**:

- `COUNT(*)`: number of rows.
- `COUNT(column)`: number of non-NULL values in that column.
- `COUNT(DISTINCT column)`: number of distinct non-NULL values.
- `SUM(column)`: total of all values.
- `AVG(column)`: arithmetic mean.
- `MIN(column)` / `MAX(column)`: smallest / largest value.

Without `GROUP BY`, an aggregate function operates on **all rows** as one group.

Demo 1: Aggregation without GROUP BY (whole-table summary)

These queries treat the entire table as a single group:

```
-- How many students are there in total?
SELECT COUNT(*) AS total_students FROM student;

-- What is the average instructor salary across all departments?
SELECT AVG(salary) AS avg_salary FROM instructor;

-- What is the highest and lowest salary?
SELECT MIN(salary) AS min_sal, MAX(salary) AS max_sal FROM instructor;
```

Each query returns exactly **one row** because there is no `GROUP BY`.

Demo 2: GROUP BY — aggregation per group

Key Concept: What does GROUP BY do?

`GROUP BY column` splits the rows into groups where every row in a group has the same value for that column. The aggregate function then runs **once per group** instead of once for the whole table.

Count how many students are in each department:

```
SELECT dept_name, COUNT(*) AS num_students
FROM student
GROUP BY dept_name
ORDER BY num_students DESC;
```

Average instructor salary per department:

```
SELECT dept_name, AVG(salary) AS avg_salary
FROM instructor
GROUP BY dept_name
ORDER BY avg_salary DESC;
```

The GROUP BY rule

When you use `GROUP BY`, every column in the `SELECT` list must be either:

1. listed in the `GROUP BY` clause, **or**
2. wrapped in an aggregate function (`COUNT`, `SUM`, etc.).

Violating this rule causes an error: `ORA-00979: not a GROUP BY expression`.
Example of the error:

```
-- WRONG: 'name' is not in GROUP BY and not aggregated
SELECT dept_name, name, COUNT(*)
FROM student
GROUP BY dept_name;
```

Demo 3: Multiple aggregations in one query

You can compute several aggregates at once:

```
SELECT dept_name,
       COUNT(*) AS num_instructors,
       MIN(salary) AS min_sal,
       MAX(salary) AS max_sal,
       ROUND(AVG(salary), 2) AS avg_sal
FROM instructor
GROUP BY dept_name
ORDER BY avg_sal DESC;
```

`ROUND(value, 2)` rounds the average to 2 decimal places for readability.

Demo 4: GROUP BY with JOIN

You can combine `GROUP BY` with a `JOIN` to aggregate across related tables. For each student, count how many courses they have taken:

```
SELECT s.ID, s.name, COUNT(*) AS courses_taken
FROM student s
JOIN takes t ON t.ID = s.ID
GROUP BY s.ID, s.name
ORDER BY courses_taken DESC, s.name;
```

Notice that both `s.ID` and `s.name` appear in `GROUP BY` because both appear in the `SELECT` list.

Demo 5: GROUP BY multiple columns

You can group by more than one column. Count how many sections were offered per semester and year:

```
SELECT semester, year, COUNT(*) AS num_sections
FROM section
GROUP BY semester, year
ORDER BY year, semester;
```

Each unique `(semester, year)` combination becomes its own group.

Part 2: Filtering Groups with HAVING

Key Concept: WHERE vs HAVING

- `WHERE` filters **individual rows** *before* grouping.
- `HAVING` filters **entire groups** *after* grouping and aggregation.

Think of the execution order as:

1. `FROM / JOIN` — get the raw rows
2. `WHERE` — discard rows that don't match
3. `GROUP BY` — form groups from remaining rows
4. `HAVING` — discard groups that don't match
5. `SELECT` — compute output columns
6. `ORDER BY` — sort the final result

Demo 6: HAVING with COUNT

Find departments that have **more than 2 students**:

```
SELECT dept_name, COUNT(*) AS num_students
FROM student
GROUP BY dept_name
HAVING COUNT(*) > 2
```

```
ORDER BY num_students DESC;
```

`HAVING COUNT(*) > 2` removes groups where the count is 2 or less. Only groups (departments) with 3 or more students survive.

Demo 7: HAVING with AVG

Find departments where the average instructor salary exceeds 80000:

```
SELECT dept_name, ROUND(AVG(salary), 2) AS avg_salary
FROM instructor
GROUP BY dept_name
HAVING AVG(salary) > 80000
ORDER BY avg_salary DESC;
```

Common mistake: using WHERE instead of HAVING

```
-- WRONG: aggregate functions cannot appear in WHERE
SELECT dept_name, AVG(salary)
FROM instructor
WHERE AVG(salary) > 80000
GROUP BY dept_name;
```

This will cause an error: `ORA-00934: group function is not allowed here.`
Rule: if your condition involves an aggregate function, it must go in `HAVING`, not `WHERE`.

Demo 8: WHERE + GROUP BY + HAVING together

Find departments where the average salary of instructors earning above 50000 exceeds 80000:

```
SELECT dept_name,
       COUNT(*) AS num_instructors,
       ROUND(AVG(salary), 2) AS avg_salary
FROM instructor
WHERE salary > 50000
GROUP BY dept_name
HAVING AVG(salary) > 80000
ORDER BY avg_salary DESC;
```

Step by step:

1. `WHERE salary > 50000` removes low-salary rows *first*.
2. `GROUP BY dept_name` groups the remaining rows by department.
3. `HAVING AVG(salary) > 80000` keeps only groups whose average (of the remaining rows) exceeds 80000.

Part 3: Putting It All Together

Key Concept: The full query structure

A complete query can use all of these clauses (in this order):

```
SELECT   columns / aggregates
FROM     table(s) / JOIN
WHERE    row-level filter
GROUP BY grouping columns
HAVING   group-level filter
ORDER BY sorting
```

Demo 9: JOIN + GROUP BY + HAVING

For each course, count how many students have taken it. Only show courses taken by more than 3 students:

```
SELECT c.course_id, c.title, COUNT(DISTINCT t.ID) AS num_students
FROM course c
JOIN takes t ON t.course_id = c.course_id
GROUP BY c.course_id, c.title
HAVING COUNT(DISTINCT t.ID) > 3
ORDER BY num_students DESC;
```

`COUNT(DISTINCT t.ID)` counts unique student IDs (a student who retook a course is counted once).

Part 4: More Realistic Use-Cases of GROUP BY

Demo 10: Finding the maximum per group

Find the highest salary in each department, together with the department name:

```
SELECT dept_name, MAX(salary) AS max_salary
FROM instructor
GROUP BY dept_name
ORDER BY max_salary DESC;
```

Follow-up question (think about this): how would you also display the *name* of the instructor who earns that maximum salary? (Hint: this is harder than it looks — you would need a subquery or a self-join, which we will cover in a future Side Activity.)

Demo 11: COUNT with and without DISTINCT

Compare `COUNT(*)` vs `COUNT(DISTINCT ...)` when counting enrollments per department:

```
-- COUNT(*): counts every enrollment row (duplicates if a student
-- took multiple courses from the same department)
SELECT c.dept_name, COUNT(*) AS total_enrollments
FROM takes t
JOIN course c ON c.course_id = t.course_id
GROUP BY c.dept_name
ORDER BY total_enrollments DESC;
```

```
-- COUNT(DISTINCT t.ID): counts unique students
SELECT c.dept_name, COUNT(DISTINCT t.ID) AS unique_students
FROM takes t
JOIN course c ON c.course_id = t.course_id
GROUP BY c.dept_name
ORDER BY unique_students DESC;
```

The first query counts total enrollment rows; the second counts distinct students. Notice the difference: Comp. Sci. has many enrollment rows because some students take multiple CS courses, but fewer unique students.

Part 5: Self Exercises

All questions use the **University schema**.

GROUP BY questions

Q1. (GROUP BY) For each department, find the total number of courses offered by that department. Output: `dept_name, num_courses`. Order by `num_courses` descending.

Q2. (GROUP BY + JOIN) For each instructor, find how many course sections they have taught (i.e., count rows in `teaches`). Output: `ID, name, num_sections`. Order by `num_sections` descending, then `name`.

Q3. (GROUP BY multiple columns) For each `(semester, year)` combination, find the total number of students who enrolled in at least one course (count distinct student IDs from `takes`). Output: `semester, year, num_students`. Order by `year`, then `semester`.

Q4. (GROUP BY + aggregate) For each department, find the lowest instructor salary. Output: `dept_name, min_salary`. Order by `min_salary`.

GROUP BY + HAVING questions

Q5. (HAVING + COUNT) Find all departments that offer more than 3 courses. Output: `dept_name, num_courses`. Order by `num_courses` descending.

Q6. (HAVING + AVG) Find all departments where the average instructor salary is below 70000. Output: `dept_name, avg_salary` (rounded to 2 decimal places). Order by `avg_salary`.

Q7. (WHERE + GROUP BY + HAVING) Among all course sections offered in year 2017 or later, find courses that have been offered more than once (i.e., appear in the `section` table more than once). Output: `course_id, num_offerings`. Order by `num_offerings` descending, then `course_id`.

Hint: Use `WHERE year >= 2017` to filter rows first, then `GROUP BY course_id` and `HAVING COUNT(*) > 1`.

Q8. (JOIN + GROUP BY + HAVING) Find students who have taken more than 3 courses in total. Display their ID, name, and the number of distinct courses taken. Output: `ID, name, num_courses`. Order by `num_courses` descending.

Hint: Join `student` with `takes`, group by `(ID, name)`, and use `HAVING COUNT(DISTINCT course_id) > 3`.

End of Side Activity 9.